

Federated learning Resnet-50

Sami Serhan, Abbas Ismail

February 28, 2026

Abstract

Federated learning is an emerging paradigm designed to address one of the most sensitive barriers in modern machine learning: the need to collaborate on model training without ever exchanging the underlying data. In many domains, such as healthcare organizations guarding patient records, companies protecting proprietary codebases, or any institution handling private personal information, centralizing raw datasets is simply not an option. Traditional approaches require aggregating data on a single server, which creates legal, ethical and security risks.

Federated learning overcomes this challenge by inverting the usual workflow. Instead of collecting data, a global model is distributed to each data holder (“client”) and trained locally on that client’s private dataset. Only the resulting model updates – never the raw observations themselves – are sent back to a coordinating server, which aggregates them to improve the shared model. Through repeated rounds of this exchange, the system converges toward a high-quality model while the data remain sequestered behind the owners’ firewalls.

The goal of this project is to demonstrate that federated learning can be applied in practice and to quantify its effectiveness on a well-known benchmark. We simulate a four-site federated system using the CIFAR-10 image classification dataset, with each site training a ResNet-50 network on its local partition. The server employs standard federated averaging to merge client models, and we monitor performance across training rounds. A conventionally trained, centralized ResNet-50 on CIFAR-10 typically reaches around 93–95% accuracy, and our federated implementation converges close to this level, achieving a strong global accuracy of 90% without ever pooling client data. This illustrates how federated learning enables collaborative model development while preserving data privacy.

1 Architecture

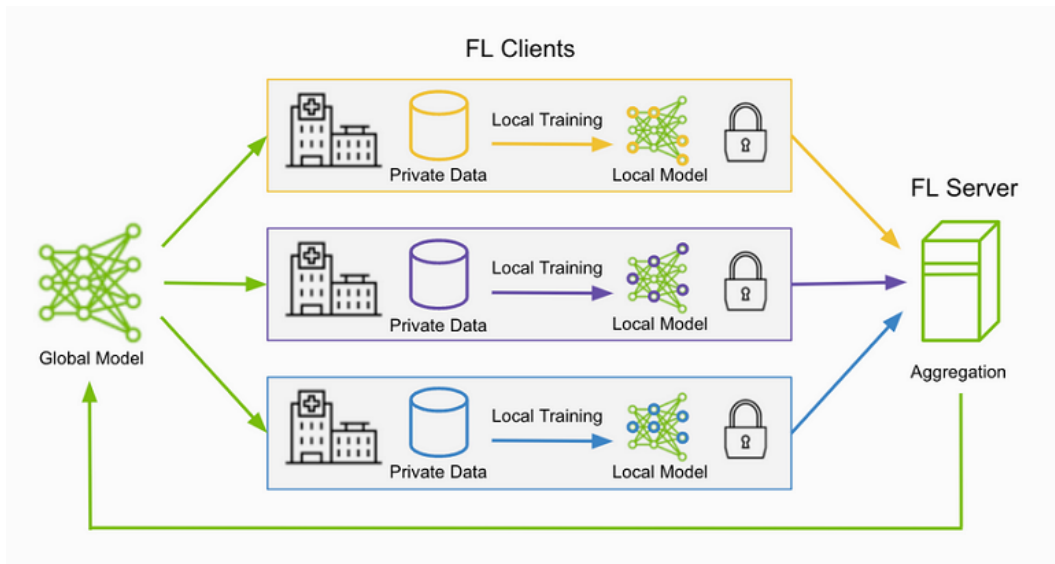


Figure 1: System architecture for the federated learning experiment.

1.1 Data Distribution

The CIFAR-10 Dataset contains 50,000 images from 10 categories: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck, we want to split this data over a number of clients. To ensure robust training the data splits should be deterministic at each run, and each client should receive an equal number of data instances from each label.

The splitting process:

- Download and load the datasets.
- Deterministically split training data equally among clients.

```
1 def split_cifar10_data(  
2     data_path: str, num_clients: int = 4, batch_size: int = 64  
3 )  
4 ...  
5     labels = np.array(train_dataset.targets)  
6     client_indices = [[] for _ in range(num_clients)]  
7  
8     for label in range(10):  
9         idx = np.where(labels == label)[0] # For each label get it's indices  
10        idx = np.sort(idx) # determinism  
11        splits = np.array_split(idx, num_clients)  
12        for i, s in enumerate(splits):  
13            client_indices[i].extend(s.tolist())
```

This also ensure that each client will get the same amount of data for each label for more robust training.

- Create Data Loaders for each client with the provided batch size.

Since the test data is small (10,000 samples) we will treat it as a shared test data for all of the clients. Note that this will not affect the federated learning concept since yet each client will maintain it's private share from the training data.

1.2 Server

The server-side implementation consists of creating Controllers, which define server-side aggregation workflow and the interaction logic with clients.

```
1 initial_model = get_resnet50_model()  
2 Agg_method = aggregation_methods[aggregation_method.lower()]  
3 job = Agg_method(  
4     name=aggregation_method.lower(),  
5     n_clients=n_clients,  
6     num_rounds=num_rounds,  
7     initial_model=initial_model,  
8 )  
9  
10 for i, (client_id, client_config) in enumerate(clients.items()):  
11     executor = ScriptRunner(  
12         script="federated_learning/client.py",  
13         script_args=(f"--client_id site-{i+1} --num_clients {n_clients} "  
14                     f"--lr {client_config.lr} --lr_min {client_config.lr_min} "  
15                     f"--batch_size {client_config.batch_size} --epochs  
16                     ↪ {client_config.epochs} "  
17                     f"--use_lr_scheduler {str(client_config.use_lr_scheduler)} "  
18                     f"--workspace_path {run_path} --data_path {data_path}"),  
19     )  
20     job.to(executor, f"site-{i + 1}") # Assign executors (`training script`) for each  
21     ↪ client.
```

Explanation: Aggregation defines how the server will deal with the clients returned results (e.g: weights). In our case we use FedAvg - federated averaging. The algorithm is very simple:

- Each client i trains the current global model on its local data and returns its updated parameters w_i .
- The server aggregates the received model updates from clients by performing a weighted average.
- The averaged model becomes the global model sent to clients in the next round.

Lastly NVFLARE offer two ways to run the server locally and we support both of them:

1. **FL Simulator**, a handy runtime tool that allows researchers to quickly test-run a federated job in a local development environment.

```
1 job.simulator_run(workspace=run_path)
```

2. **Proof-of-Concept (PoC)**, a suite of command-line tools that allow FL developers to run a provisioned project in a sandbox environment on a single PC.

```
1 job.export_job(run_path)
```

And then you can follow this commands to run using the **PoC**:

- `nvflare poc prepare -n number-of-clients`
- `nvflare poc prepare-jobs-dir -j run_path`
- `nvflare poc start -ex admin@nvidia.com`
- Start new wsl terminal for admin console, run: `nvflare poc start -p admin@nvidia.com`
- `submit_job fedavg`

For more information of how to enable **Simulator** mode or **PoC** mode see Section 2.1.1 and 1.3.

1.3 PoC

In the proof-of-concept (PoC) setup using **NVFLARE**, provisioning is a crucial step that defines and configures all participants involved in the federated learning project. Unlike simulated environments where participants are automatically generated, a PoC that resembles real-world deployment requires an explicit definition of identities, roles, and access privileges.

Provisioning establishes trusted identities for each participant and generates the required startup kits that enable secure communication within the federated system. In the PoC, this process ensures that all components—server, clients, and administrators—can authenticate each other and operate within a secure and controlled environment.

1.3.1 Participant Types

NVFLARE distinguishes between different types of participants, each assigned a clearly defined role. These participants can be categorized into two main groups: *sites* and *users*.

Sites. Sites represent computing systems that execute federated learning workloads. In the PoC, the following site types are defined:

- **Server:** The central coordinator of the federated learning process. The server is responsible for orchestrating communication rounds, distributing the global model, aggregating client updates, and managing the overall training workflow.
- **Client:** A participant that performs local training on its private dataset. Each client receives the global model from the server, updates it using local data, and sends the updated parameters back for aggregation.

Both server and clients run FLARE applications and therefore require properly provisioned startup kits containing identity credentials and security configurations.

Users. Users represent human participants who interact with the federated learning system. In the PoC, the primary user role is:

- **Admin:** The administrator is responsible for managing, monitoring, and controlling the federated learning project. The admin has elevated privileges that allow querying system status, submitting jobs, and overseeing the training process.

Unlike sites, users do not perform model training. Instead, they authenticate into the system to supervise and manage operations according to their assigned access privileges.

1.4 Client

Clients implement Executors which handle the training logic that happens locally on each client site.

1.4.1 Receive the data

Since we are working in one computer and our dataset (CIFAR-10) is joined, each client needs a way to figure out what share of data he gets for the training, so we introduced this function:

```
1 def get_client_data_loader(  
2     client_id: str, batch_size: int, num_clients: int, data_path: str  
3 ) -> tuple[DataLoader, DataLoader]:  
4     client_loaders, test_loader = split_cifar10_data(  
5         data_path=data_path, num_clients=num_clients, batch_size=batch_size  
6     )  
7     indx = int(re.search(r"site-(\d+)", client_id).group(1)) - 1  
8     return client_loaders[indx], test_loader # client training set and the shared test  
        ↪ set
```

This function takes the number of clients and the client ID and it knows which data part needed to be returned accordingly, we also supply the data path so all of the clients know where the global data lives to avoid downloading the full data again and again per client.

1.4.2 Training and evaluating

Each client participates in the federated optimization process by performing local training on its private dataset after receiving the current global model from the server. Upon reception, the global model is first evaluated on the client's local test set to compute accuracy and average loss. These metrics are logged for monitoring the global model's performance across communication rounds.

```
1     summary_writer.add_scalar(  
2         tag="global/global_model_accuracy",  
3         scalar_value=accuracy,  
4         global_step=input_model.current_round + 1,  
5     )  
6     summary_writer.add_scalar(  
7         tag="global/global_model_loss",  
8         scalar_value=avg_loss,  
9         global_step=input_model.current_round + 1,  
10    )
```

Optimizer. Local training is performed using **Stochastic Gradient Descent (SGD)** with momentum and weight decay regularization. Specifically, the optimizer is configured with:

- Learning rate: lr
- Momentum: 0.9
- Weight decay: 5×10^{-4}

The parameter update rule at iteration t is given by:

$$v_t = 0.9 v_{t-1} + \nabla \mathcal{L}(\theta_t),$$

$$\theta_{t+1} = \theta_t - lr \cdot v_t - lr \cdot 5 \times 10^{-4} \cdot \theta_t,$$

where θ_t denotes the model parameters and $\mathcal{L}$ is the training loss.

Learning Rate Scheduler. A **Cosine Annealing Learning Rate Scheduler** is applied across federated rounds. The learning rate at round t is computed as:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min}) \left(1 + \cos \left(\frac{t}{T_{\max}} \pi \right) \right),$$

where:

- η_0 is the initial learning rate,
- $\eta_{\min}$ is the minimum learning rate,
- $T_{\max}$ is the total number of federated rounds,
- t is the current round index.

```

1     scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
2         optimizer,
3         T_max=input_model.total_rounds,
4         last_epoch=input_model.current_round - 1,
5         eta_min=lr_min,
6     )
7     scheduler.step()

```

Note that the scheduler updates the learning rate per round not per epoch as usually done. This scheduler ensures smooth and non-linear decay of the learning rate over communication rounds, which proved to enhance convergence stability in the training we did.

Local Training Procedure. After loading the received global parameters into the ResNet-50 model, the client performs training for a fixed number of local epochs. For each mini-batch, the following steps are executed:

1. Forward propagation.
2. Loss computation,
3. Backpropagation.
4. Parameter update using SGD.

For each training epoch we record the training loss to the tensor-board so we can check it later.

```

1     training_loss = running_loss / len(train_loader)
2     # Step associated to the current round
3     training_step = input_model.current_round * epochs + (epoch + 1)
4     summary_writer.add_scalar(
5         tag="local_training_loss",
6         scalar_value=training_loss,
7         global_step=training_step,
8     )

```

Post-training Evaluation. After completing local updates, the trained model is evaluated again on the local test set to compute overall accuracy, per-class accuracy, and average loss. These metrics are logged/recorded and the updated parameters are sent back to the server for aggregation in the next communication round.

```

1     output_model = flare.FLModel(
2         params=resnet.cpu().state_dict(), # Client trained local model
3         metrics={"accuracy": accuracy}, # Global model accuracy for model selection
4         meta={"NUM_STEPS_CURRENT_ROUND": steps},
5     )
6
7     flare.send(output_model)

```

1.5 Model Definition

The experiment uses a ResNet variant implemented from pytorch. The shared model factory is:

```

1     def get_resnet50_model():
2         model = ResNet() # custom ResNet with 10 output classes
3         return model

```

The architecture is adapted for CIFAR-10 (10 output classes).

1.6 Best model selection

When we send the accuracy to the server for the global model nvflare has a model selection mechanism that saves the best model through the run so the users can get both of the last global model and the best model based on the metric sent in our case it's the test accuracy.

Example:

```

1     best_model_path = os.path.join(
2         run_path, "server/simulate_job/app_server/best_FL_global_model.pt"
3     )
4     last_model_path = os.path.join(
5         run_path, "server/simulate_job/app_server/FL_global_model.pt"
6     )

```

1.7 All together

1. The server distributes the `initial_model` and starts the federated job.
2. Each client receives global parameters, evaluates them, trains locally, and returns updated weights and metrics.
3. The server aggregates updates to the new global model and repeats for `num_rounds`.
4. No raw data leaves any client; only model parameters and metadata are exchanged.

2 Configuration

Our federated learning system is configured using one YAML file. The configuration controls server-side aggregation settings and client-side training parameters. This configuration makes it easier to run the federated learning runs with different parameters.

2.1 Configuration Structure

The configuration file defines both the server-side and client-side parameters used during federated training. It consists of two main components:

- **Server settings:** Control the global training process.
- **Client settings:** Define local training hyperparameters for each participating client.

2.1.1 Server Configuration

Table 1: Server configuration parameters

Parameter	Type	Description
num_rounds	int	Number of federated learning communication rounds
aggregation_method	string	Aggregation strategy for client models (default: <code>fedavg</code>)
poc_mode	bool	when enabled we will run with PoC (default: <code>False</code>)

Supported Aggregation Methods

- `fedavg` – Federated Averaging (default)

2.1.2 Client Configuration

Each client can be configured independently using the parameters listed in Table 2.1.2.

Table 2: Client configuration parameters

Parameter	Type	Default	Description
lr	float	0.03	Learning rate for local training
epochs	int	6	Number of local epochs per round
batch_size	int	64	Mini-batch size
use_lr_scheduler	bool	true	Enable cosine annealing scheduler
lr_min	float	0.001	Minimum learning rate when scheduler is used

2.2 Configuration Example

```
server:
  num_rounds: 10
  aggregation_method: fedavg
  poc_mode: true
  clients:
    site-1:
      lr: 0.03
      epochs: 5
      batch_size: 64
      use_lr_scheduler: true
      lr_min: 0.001
    site-2:
      lr: 0.03
      epochs: 6
      batch_size: 64
      use_lr_scheduler: true
      lr_min: 0.001
    site-3:
      lr: 0.01
      epochs: 3
      batch_size: 64
      use_lr_scheduler: true
      lr_min: 0.001
    site-4:
      lr: 0.1
      epochs: 6
```

```

batch_size: 128
use_lr_scheduler: false
lr_min: 0.01

```

Default Configuration If no client-specific parameters are provided, default values are applied automatically:

```

server:
  num_rounds: 100
  aggregation_method: fedavg
  clients:
    client_1: {}
    client_2: {}
    client_3: {}
    client_4: {}

```

3 Results

In our run we were able to get to 90% with 4 clients, here are the training metrics and results:

3.1 Global Model

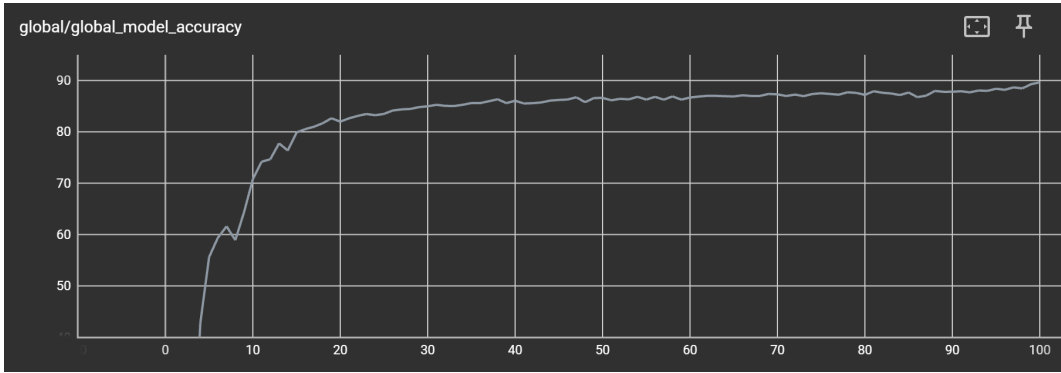


Figure 2: Global model test accuracy across federated training rounds. The x-axis represents the communication round, and the y-axis denotes classification accuracy.

Figure 2 shows the evolution of the global model’s test accuracy over federated communication rounds. As observed, the accuracy steadily increases and converges to approximately 90%. This behavior indicates that the aggregation process successfully integrates knowledge from participating clients, leading to progressive improvement of the shared global model.

Figure 3 presents the test loss of the global model across training rounds. The consistent decrease in test loss throughout the training process is a positive sign, as it indicates that the model’s predictions are becoming increasingly well-calibrated with respect to the ground truth labels.

The simultaneous increase in accuracy and decrease in test loss suggests that the federated training procedure is converging properly without evident signs of instability or divergence. If the loss were decreasing while accuracy stagnated or dropped, it could indicate overfitting or poor calibration. However, in this case, both metrics evolve consistently, supporting the effectiveness of the chosen optimizer, learning rate schedule, and aggregation strategy.

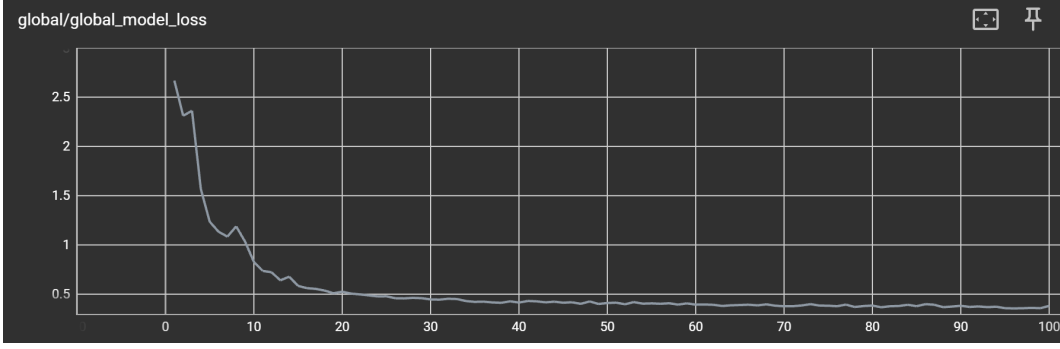


Figure 3: Global model test loss across federated training rounds. The x-axis represents the communication round, and the y-axis denotes the average test loss.

3.2 Site-1 results

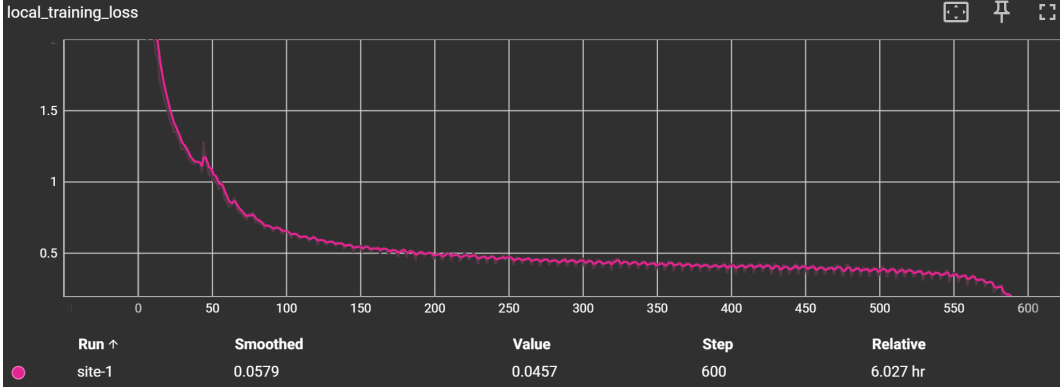


Figure 4: Global model test loss across federated training rounds. The x-axis represents the communication round, and the y-axis denotes the average test loss.

Figure 4 We can see that the training loss is decreasing and reaches below 0.05.

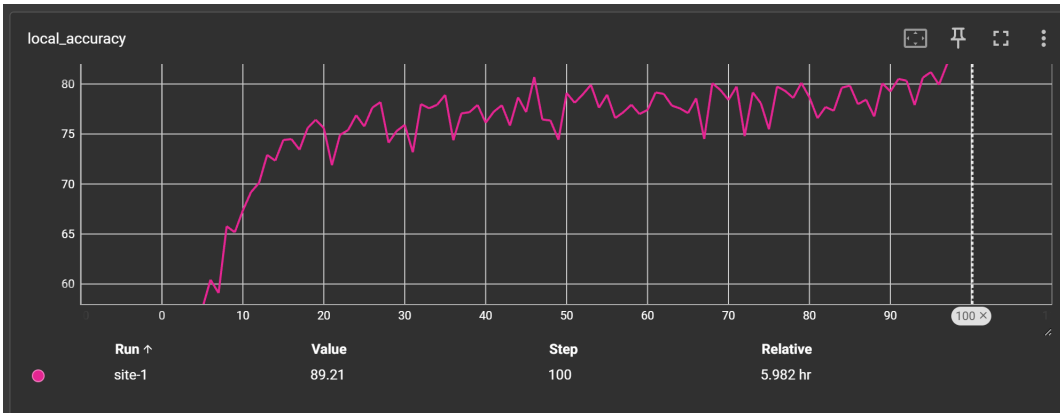


Figure 5: Site 1 local test accuracy across federated training rounds. The x-axis represents the communication round, and the y-axis is accuracy.

Figures 5 and 6 illustrate the local performance of the first client (Site 1) throughout the federated training process. The accuracy curve shows a steady improvement across communication rounds, indicating that the client effectively benefits from the aggregated global updates.

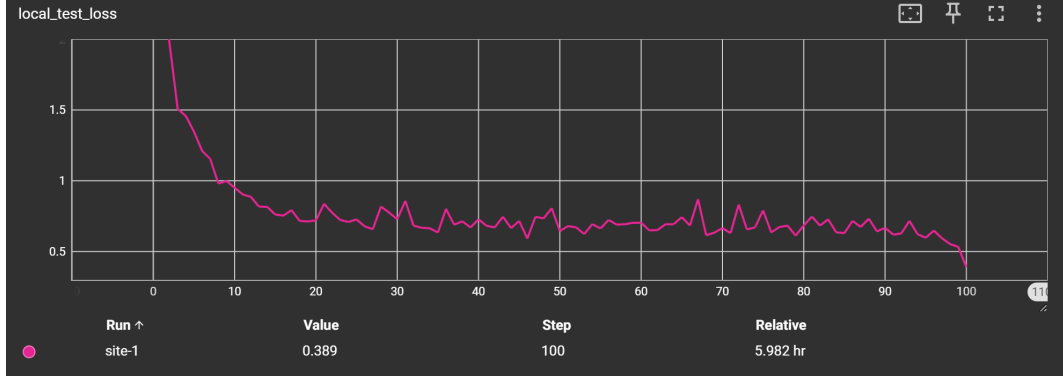


Figure 6: Site 1 local test loss across federated training rounds. The x-axis represents the communication round, and the y-axis denotes the average test loss.

At the same time, the test loss consistently decreases, which is a strong indicator of stable convergence and improved generalization on the client’s local data distribution. The simultaneous increase in accuracy and decrease in loss suggests that the model is not only making more correct predictions and no overfitting.

Overall, the results for Site 1 demonstrate that the federated learning procedure provides meaningful performance gains at the client level, without signs of instability or degradation across rounds.

3.2.1 Class-wise Accuracy

As a nice-to-have, we include class-wise accuracy per site, this will be our indicator of how each site performs on different classes we will show the results for site-1. To view the results for all of the sites on tensorboard just run this command:

```
tensorboard --logdir=results/tensorboard_logs/ --bind_all
```

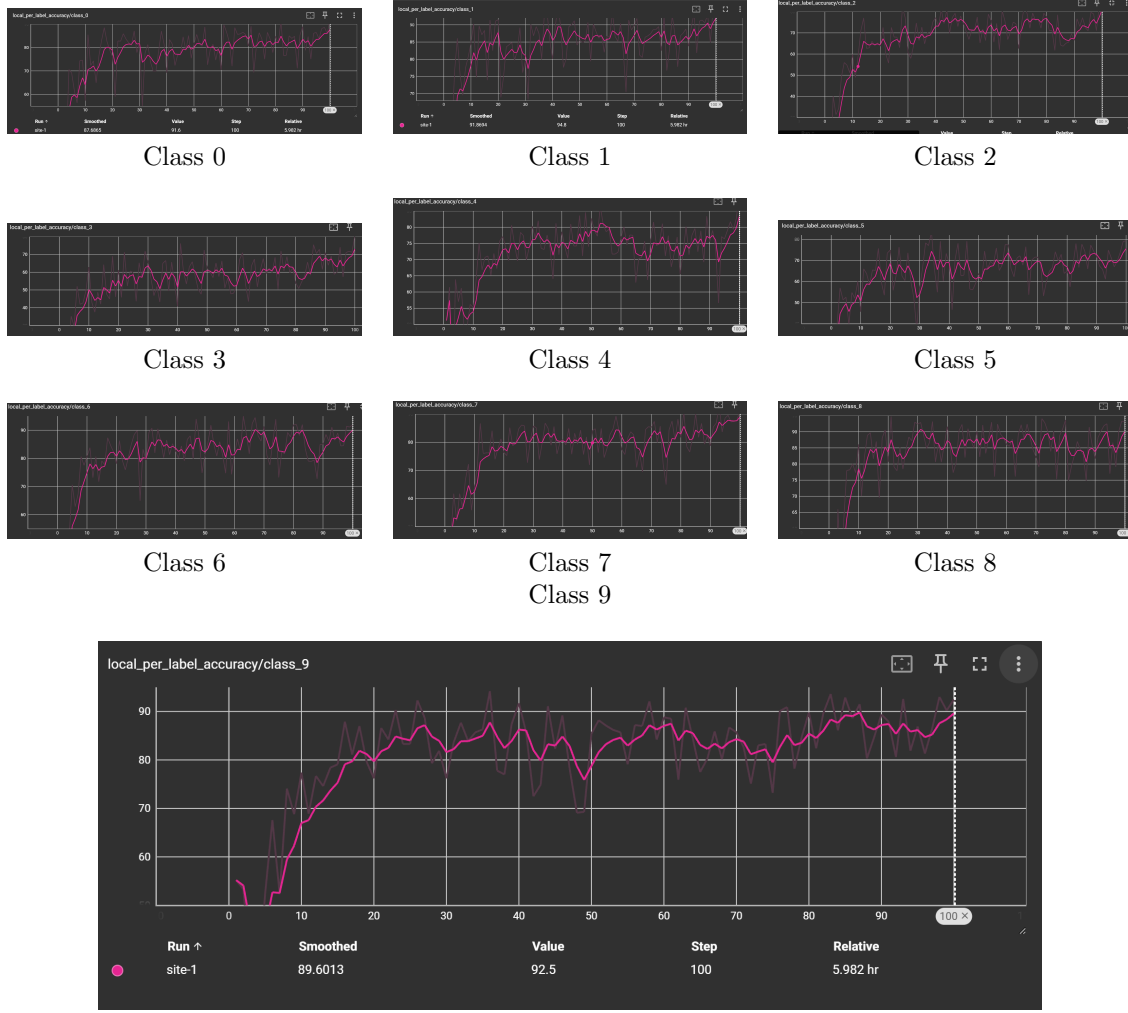


Figure 7: Per-class test accuracy for Site 1 across federated training rounds. Each subplot shows the accuracy evolution for a specific class.

3.3 Used Configuration

After conducting multiple experimental runs with different hyperparameter configurations, we converged on a unified setup applied consistently across all participating clients. Empirically, we observed that using identical training parameters for all sites resulted in more stable convergence and improved overall performance compared to heterogeneous configurations. In our setting, maintaining consistent learning rates, batch sizes, and numbers of local epochs across clients led to better coordination during the aggregation process.

Furthermore, the learning rate scheduler played a crucial role in achieving strong final performance. In particular, the cosine annealing schedule enabled the learning rate to gradually decrease over the course of federated rounds, allowing the model to perform larger updates during the early stages of training and finer adjustments near convergence. This proved especially important in the later rounds, where a smaller learning rate was necessary for stable optimization.

To validate this observation, we conducted ablation experiments by disabling the scheduler (i.e. setting `use_lr_scheduler` to `false`). In this case, the final global accuracy dropped to approximately 80%, confirming that learning rate decay significantly improves convergence quality and final model performance in our federated learning setup.

```
{
  server: {
    num_rounds: 100,
```

```

aggregation_method: fedavg,
clients: {
  client_1: {
    lr: 0.03,
    epochs: 6,
    batch_size: 64,
    use_lr_scheduler: true,
    lr_min: 0.001
  },
  client_2: {
    lr: 0.03,
    epochs: 6,
    batch_size: 64,
    use_lr_scheduler: true,
    lr_min: 0.001
  },
  client_3: {
    lr: 0.03,
    epochs: 6,
    batch_size: 64,
    use_lr_scheduler: true,
    lr_min: 0.001
  },
  client_4: {
    lr: 0.03,
    epochs: 6,
    batch_size: 64,
    use_lr_scheduler: true,
    lr_min: 0.001
  }
}
}
}
}

```